

Студент: Чернышева Елизавета Сергеевна
Курс: 3
Группа: Z33433
2025 г.

Лабораторная работа №4

C++ PROCESSES / THREADS

1 Цель работы

Познакомить студента с принципами параллельных вычислений. Составить несколько программ в простейшими вычислительными действиями, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

2 Задачи

1. [C++ SEQUENCE] Последовательные вычисления

Требуется последовательно выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1.

Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек.

2. [C++ THREADS] Параллельные вычисления через потоки

Требуется параллельно (насколько возможно с помощью потоков) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1.

Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек. в разбивке по шагам вычислений 1, 2 и 3.

3. [C++ PROCESS] Параллельные вычисления через процессы

Требуется параллельно (насколько возможно с помощью процессов) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2

– результат вычислений 1.

Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек. в разбивке по шагам вычислений 1, 2 и 3

4. **[LOG] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)**

Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

3 Решение

1. **[C++ SEQUENCE] Последовательные вычисления**

В папке src проекта созданы отдельные файлы func1.cpp, func2.cpp, func3.cpp с функциями. Последовательные вычисления производятся сразу в двух файлах - sequentialCalc.cpp и sequentialCalcOpt.cpp. В первом отключена оптимизация компилятора, поскольку т.к. уравнения можно сильно упростить, компилятор будет пытаться избавиться от лишних вычислений (например, $x^2 - x^2$). Отключение оптимизации реализовано с помощью команды `<volatile double dummy = 0>`. Созданная переменная dummy - мусорная, она не несет никакой смысловой нагрузки. Она включена в цикл, где ее значение постоянно меняется, поэтому цикл не может быть удален при попытке оптимизации компилятором. Соответственно, эта хитрость мешает оптимизации. Во втором файле не стали принудительно отключать оптимизацию, чтобы посмотреть, как сильно это повлияет на скорость вычислений.

2. **[C++ THREADS] Параллельные вычисления через потоки**

За параллельные вычисления через потоки отвечает файл threadsCalc.cpp. Здесь создаются потоки только для функций func1 и func2, т.к. они могут быть вычислены независимо. Для третьей функции func3 нам понадобятся результаты первых двух, поэтому она не может быть вычислена одновременно с ними. Поэтому мы создаем внутри основной функции два потока с циклами по итерациям, в которых вычисляются первые две функции. Основная программа не может продолжить свою работу, пока эти потоки не завершатся, для этого используется команда `<thread*.join()>`. Это нужно для того, чтобы данные функций корректно записались и ничего не было утеряно. В этой программе было реализовано хранение каждого результата вычисления функций на каждой итерации в виде вектора. По сути, здесь это то же самое, как и создание переменной dummy, циклы не будут уничтожаться при оптимизации компилятором программы во избежание ненужных вычислений, и мы сможем корректно измерить время работы. В векторах данные в памяти хранятся непрерывно, что ускоряет доступ к ним, в отличие от других контейнеров. Однако, создание векторов требует резервирования памяти (≈ 2.4 МБ ненужной памяти), что не очень эффективно. Второй вариант реализации - можно было подобно первому пункту создать переменную dummy и изменять ее значение в цикле, чтобы он не удалялся. После того, как потоки завершили свою работу, мы, используя значения первых двух функций, записанные в вектора, вычисляем третью функцию.

3. [C++ PROCESS] Параллельные вычисления через процессы

Здесь идея очень похожа на реализацию параллельных вычислений через потоки. Создаются два дочерних процесса через `fork()`, наследующие часть окружения родителя, для первых двух функций, внутри которых в цикле происходит их вычисление и запись результатов в вектор. Каждому процессу присвоен свой PID (Process ID). Для процессов создаются каналы `pipe` (IPC - межпроцессорное взаимодействие), по которым они "общаются" друг с другом. В нашем коде для дочерних процессов открыт канал только на запись, закрыв его для чтения. После записи данных в канал, мы должны его закрыть для записи. Родительский же процесс использует каналы только для чтения, закрывая их для записи. Результаты чтения записываются в созданные уже в родительском процессе вектора, отдельно от дочерних процессов. Как и в потоках, мы используем команду `<waitpid(pid*, nullptr, 0)>` для ожидания завершения дочерних процессов перед продолжением выполнения родительского. Функция `waitpid()` приостанавливает выполнение текущего (родительского) процесса до тех пор, пока не завершится указанный дочерний процесс. Это необходимо для корректного завершения дочерних процессов, предотвращения "зомби" процессов (когда процесс завершился, но запись о нём остаётся в системе), синхронизации между процессами. `pid*` — PID дочернего процесса, которого нужно ожидать, `nullptr` — означает, что статус завершения процесса не сохраняется (не интересует), `0` — флаги ожидания (обычное блокирующее ожидание). Далее уже в отдельном цикле вычисляется третья функция.

4 Заключение

Были получены следующие результаты:

```
Последовательные вычисления (без оптимизации компилятора):
Iterations: 10000, Time: 4.8221e-05 sec
(Dummy: 50000)
-----
Последовательные вычисления (без оптимизации компилятора):
Iterations: 100000, Time: 0.00048338 sec
(Dummy: 500000)
-----
Последовательные вычисления (с оптимизацией компилятора):
Iterations: 10000, Time: 4.4483e-05 sec
-----
Последовательные вычисления (с оптимизацией компилятора):
Iterations: 100000, Time: 0.00044566 sec
-----
Параллельные вычисления через потоки:
Iterations: 10000
Step 1 (formula1 & formula2 parallel): 0.000177965 sec
Step 2 (formula3 sequential): 7.2206e-05 sec
Total time: 0.000250532 sec
-----
Параллельные вычисления через потоки:
Iterations: 100000
Step 1 (formula1 & formula2 parallel): 0.000815285 sec
Step 2 (formula3 sequential): 0.000463463 sec
Total time: 0.00127904 sec
-----
Параллельные вычисления через процессы:
Iterations: 10000
Step 1 (formula1 & formula2 parallel): 0.000534116 sec
Step 2 (formula3 sequential): 6.4581e-05 sec
Total time: 0.000598848 sec
-----
```

Из результатов видно, что последовательные вычисления производятся намного быстрее, чем

параллельные. Вероятнее всего это связано с тем, что на создание процессов или потоков уходит довольно большая часть времени. Нам нужно сохранять большие вектора с данными, которые занимают много места в памяти из-за большого числа итераций, чтобы использовать их в дальнейшем для вычисления третьей функции, которую нельзя поместить в отдельный поток или процесс, и вычислять параллельно с первыми двумя функциями. В процессах много времени уходит на запись и чтение по каналам. Так же не удалось посчитать 100_000 итераций через процессы, что вероятнее всего вызвано переполнением канала (обычно 64КБ в Linux). Поэтому для подобных задач больше подходит последовательное вычисление. Но уже для более сложных задач больше подойдет параллельное вычисление через потоки и процессы.